POLITECHNIKA LUBELSKA

WYDZIAŁ MATEMATYKI I INFORMATYKI TECHNICZNEJ

Kierunek: Inżynieria i Analiza Danych

Narzędzia informatyczne II

Projekt z zakresu programowania

Wykonali: **Maja Darczuk, Jan Walkiewicz**

Program do Analizy Grafów

Spis treści

1 Założenia projektu **2**

2 Interfejs programu **3**
2.1 Opis Elementów . . . 3

3 Opis funkcjonalności programu **5**
3.1 `OnPaint - Tworzenie tła` . . . 5
3.2 Definicja grafu . . . 5
3.3 GraphWindow - Rysowanie grafu . . . 5
3.4 ScaleCoordinates - Skalowanie . . . 7
3.5 ModifyGraphmlFile - Modyfikacja grafu . . . 8
3.6 LoadGraph - Wczytywanie grafu . . . 10
3.7 Rysowanie grafu . . . 11
3.8 Wyświetlanie wierzchołków . . . 12
3.9 Wyświetlanie krawędzi . . . 12
3.10 Sprawdzanie typu grafu . . . 12
3.11 Sprawdzanie cykliczności grafu: . . . 12
3.12 Sprawdzanie planarności grafu: . . . 13
3.13 Wyszukiwanie najkrótszej ścieżki: . . . 13

4 Podsumowanie **15**

5 Podział pracy **15**

1 Założenia projektu

Stworzony program jest aplikacją dedykowaną do analizy grafów zapisanych w plikach GraphML. Inspiracją do jego stworzenia była potrzeba opracowania narzędzia, które w prosty i intuicyjny sposób umożliwia użytkownikom wizualizację oraz analizę podstawowych własności grafów. Program został zaprojektowany z myślą o osobach, które chcą łatwo badać strukturę grafów, identyfikować ich kluczowe cechy oraz wyciągać wnioski na podstawie przeprowadzonych analiz. Aplikacja oferuje przyjazny interfejs, który umożliwia wygodne wczytywanie plików GraphML oraz sprawne wykonywanie operacji na grafach, takich jak sprawdzanie spójności, obliczanie najkrótszej ścieżki czy wizualizacja połączeń. Dzięki tym funkcjonalnościom narzędzie znajduje zastosowanie zarówno w edukacji, jak i w pracy naukowej oraz zawodowej.

2 Interfejs programu

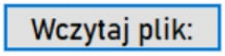
Rysunek 1: Wygląd Aplikacji

Styl wizualny:

- Interfejs 1 jest minimalistyczny i przejrzysty.
- W tle zastosowano delikatny motyw grafu, co nawiązuje do funkcji aplikacji.

2.1 Opis Elementów

Ładowanie plików:



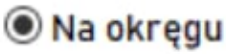
Rysunek 2: Przycisk wczytania pliku

Rysunek 3: Pole wyświetlające ścieżke do pliku

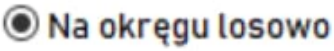
- Opcja **Wczytaj plik** 2 pozwala użytkownikowi zaimportować pliki w formacie `.graphml`.
- Obok przycisku znajduje się pole tekstowe, które wskazuje status załadowanego pliku 3 (np. „jeszcze nie wybrano...”).

Opcje wizualizacji grafu:

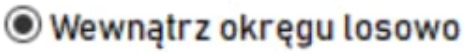
Rysunek 4: Przycisk wywołujący rysowanie grafu



Rysunek 5: Rysowanie grafu na okręgu



Rysunek 6: Rysowanie grafu na okręgu losowo



Rysunek 7: Rysowanie grafu wewnątrz okręgu losowo

- Funkcja **Rysuj Graf** 4 umożliwia wybór jednego z trzech trybów rozmieszczenia wierzchołków:
  - *Na okręgu* 5 (domyślnie zaznaczone).
  - *Na okręgu losowo* 6.
  - *Wewnątrz okręgu losowo* 7.

Przyciski akcji:

Rysunek 8: Przycisk wyświetlania wierzchołków

Rysunek 9: Przycisk wyświetlania krawędzi

Rysunek 10: Przycisk wyświetlania typu grafu

Rysunek 11: Przycisk wyświetlania najkrótszej ścieżki

- **Wyświetl wierzchołki**8: pokazuje listę wierzchołków grafu.
- **Wyświetl krawędzie**9: wyświetla krawędzie łączące wierzchołki.
- **Typ grafu** 10: określa rodzaj grafu tzn. czy jest skierowany, pełny, spójny, cykliczny, dwudzielny, planarny, czy jest drzewem lub lasem.
- **Najkrótsza ścieżka** 11: znajduje najkrótszą drogę pomiędzy wierzchołkami.

3 Opis funkcjonalności programu

3.1 OnPaint - Tworzenie tła

Funkcja ta obsługuje zdarzenie malowania (wxPaintEvent). Tworzy obiekt wxPaintDC i rysuje bitmapę tła (backgroundImage) na początkowej pozycji (0, 0).

```
void GrafyDialog::OnPaint(wxPaintEvent& event)
{
    wxPaintDC dc(this);
    dc.DrawBitmap(backgroundImage, 0, 0, true);  // Rysowanie tła
}
```

3.2 Definicja grafu

Tworzy strukturę VertexProperties do przechowywania etykiety wierzchołka oraz jego współrzędnych (x, y).

```
truct VertexProperties {
    std::string label;
    float x, y;
};
```

Tworzy strukturę EdgeProperties do przechowywania etykiety krawędzi i wagi. Definiuje typ Graph jako graf nieskierowany, używający

```
struct EdgeProperties {
    std::string label;
    int weight;
};
```

boost::adjacency_list, przechowujący właściwości wierzchołków i krawędzi.

```
typedef boost::adjacency_list<boost::vecS, boost::vecS, boost::undirectedS,
    VertexProperties, EdgeProperties> Graph;
```

3.3 GraphWindow - Rysowanie grafu

Tworzy okno (wxFrame) do rysowania grafu z określonymi parametrami. Zarejestrowane są dwie funkcje obsługi zdarzeń: OnPaint (do rysowania grafu) oraz OnClose (do zamknięcia okna). W OnPaint: Tworzy obiekt wxPaintDC i ustawia tło na białe. Wywołuje funkcję DrawGraph, która rysuje graf na podstawie trybu rysowania. W DrawGraph:
Określa parametry obszaru do rysowania (rozmiar okna, promień okręgu itp.). Na podstawie wybranego trybu rysowania (okrąg równomierny, losowy lub losowy w obrębie okręgu), oblicza pozycje wierzchołków. Rysuje krawędzie między wierzchołkami. Rysuje wierzchołki na bazie pozycji oraz wyświetla ich etykiety. Klasa odpowiedzialna za rysowanie grafu

```
class GraphWindow : public wxFrame {
```

Konstruktor klasy, inicjalizuje okno i powiązania zdarzeń

```
    GraphWindow(wxWindow* parent, const Graph& graph, int mode)
    : wxFrame(parent, wxID_ANY, "Rysowanie grafu", wxDefaultPosition, wxSize
        (800, 600)), graph(graph), drawMode(mode) {
        SetSizeHints(800, 600, 800, 600);  // Ustawienie rozmiaru okna
        Bind(wxEVT_PAINT, &GraphWindow::OnPaint, this);  // Powiązanie
            zdarzenia malowania (rysowanie)
        Bind(wxEVT_CLOSE_WINDOW, &GraphWindow::OnClose, this);  // Powiązanie
             zdarzenia zamknięcia okna
    }
```

Ustawienie rozmiaru okna

```
    SetSizeHints(800, 600, 800, 600);  // Ustawienie rozmiaru okna
```

Powiązanie zdarzenia malowania (rysowanie)

```
Bind(wxEVT_PAINT, &GraphWindow::OnPaint, this);  // Powiązanie zdarzenia
    malowania (rysowanie)
```

Powiązanie zdarzenia zamknięcia okna

```
Bind(wxEVT_CLOSE_WINDOW, &GraphWindow::OnClose, this);  // Powiązanie
    zdarzenia zamknięcia okna
```

Odwołanie do grafu, który ma zostać narysowany

```
const Graph& graph;  // Odwołanie do grafu, który ma zostać narysowany
```

Tryb rysowania grafu (np. układ okrężny, losowy itp.)

```
int drawMode;  // Tryb rysowania grafu (np. układ na okręgu, na okręgu
    losowy lub wewnątrz okręgu)
```

Obsługa zdarzenia malowania grafu

```
void OnPaint(wxPaintEvent&) {
    wxPaintDC dc(this);  // Tworzenie kontekstu rysowania (DC - Device
        Context)
    dc.SetBackground(wxBrush(wxColour(255, 255, 255)));  // Ustawienie tł
        a na biały kolor
    dc.Clear();  // Czyszczenie obszaru rysowania (wyczyszczenie tła)
    DrawGraph(dc, graph, drawMode);  // Rysowanie grafu w zależności od
        trybu
}
```

Obsługa zamknięcia okna

```
void OnClose(wxCloseEvent& event) {
    Destroy();  // Zamykanie okna aplikacji
}
```

Funkcja odpowiedzialna za rysowanie grafu

```
void DrawGraph(wxDC& dc, const Graph& graph, int mode) {
```

Pobranie szerokości okna

```
const int width = GetClientSize().GetWidth();
```

Pobranie wysokości okna

```
const int height = GetClientSize().GetHeight();
```

Obliczanie środka okna w poziomie

```
const int centerX = width / 2;
```

Obliczanie środka okna w pionie

```
const int centerY = height / 2;
```

Obliczanie promienia okręgu dla rozmieszczenia wierzchołków

```
const int radius = std::min(width, height) / 2.5;
```

Liczenie liczby wierzchołków w grafie

```
size_t vertexCount = boost::num_vertices(graph);
```

Sprawdzenie, czy graf ma wierzchołki

```
if (vertexCount == 0) return;
```

Inicjalizacja wektora do przechowywania pozycji wierzchołków

```
std::vector<wxPoint> vertexPositions(vertexCount);
```

Ustalanie pozycji wierzchołków w zależności od trybu rysowania

```
                if (mode == 1) {  // Wierzchołki rozmieszczone na okręgu z równymi odstę
                    pami
```

Obliczanie kąta dla równych odstępów

```
                    double angle = 2 * M_PI * i / vertexCount;
```

Obliczanie pozycji wierzchołka

```
                    vertexPositions[i] = wxPoint(
                    centerX + static_cast<int>(radius * cos(angle)),
                    centerY + static_cast<int>(radius * sin(angle))
                    );
```

Losowanie kątów dla wierzchołków

```
                    double angle = static_cast<double>(rand()) / RAND_MAX * 2 * M_PI;
```

Losowanie promienia i kąta dla trybu rozmieszczania losowego w obrębie okręgu

```
                    double r = static_cast<double>(rand()) / RAND_MAX * radius;
                    double angle = static_cast<double>(rand()) / RAND_MAX * 2 * M_PI;
```

Rysowanie krawędzi grafu

```
                    for (auto e : boost::make_iterator_range(boost::edges(graph))) {
```

Rysowanie krawędzi między wierzchołkami

```
                        dc.SetPen(wxPen(wxColour(74, 74, 74), 3));
                        dc.DrawLine(vertexPositions[sourceIdx], vertexPositions[targetIdx
                            ]);
```

Rysowanie wierzchołków

```
                        dc.SetBrush(wxBrush(wxColour(238, 190, 248)));
                        dc.DrawCircle(vertexPositions[index], 8);
                        dc.DrawText(vertex.label, vertexPositions[index] + wxPoint(10,
                            -10));
```

3.4 ScaleCoordinates - Skalowanie

Funkcja skalująca współrzędne do zakresu widocznego okna

```
        void ScaleCoordinates(Graph& graph, int width, int height) {
```

Inicjalizacja zmiennych dla minimalnych i maksymalnych współrzędnych

```
            float minX = std::numeric_limits<float>::max();
            float minY = std::numeric_limits<float>::max();
            float maxX = std::numeric_limits<float>::lowest();
            float maxY = std::numeric_limits<float>::lowest();
```

Przechodzenie przez wierzchołki grafu i aktualizacja minimalnych i maksymalnych współrzędnych

```
            for (auto v : boost::make_iterator_range(boost::vertices(graph))) {
                minX = std::min(minX, graph[v].x);
                minY = std::min(minY, graph[v].y);
                maxX = std::max(maxX, graph[v].x);
                maxY = std::max(maxY, graph[v].y);
            }
```

Obliczanie skali dla osi X i Y

```
            float scaleX = (width - 20) / (maxX - minX);
            float scaleY = (height - 20) / (maxY - minY);
```

Wybór mniejszej skali, aby zachować proporcje

```
        float scale = std::min(scaleX, scaleY);
```

Skalowanie współrzędnych wierzchołków grafu

```
        for (auto v : boost::make_iterator_range(boost::vertices(graph))) {
            graph[v].x = 10 + (graph[v].x - minX) * scale;
            graph[v].y = 10 + (graph[v].y - minY) * scale;
        }
```

3.5 ModifyGraphmlFile - Modyfikacja grafu

Funkcja odpowiedzialna za modyfikację pliku GraphML

```
    void ModifyGraphmlFile(const std::string& inputFilename, const std::string&
        outputFilename) {
```

Otwieranie pliku wejściowego

```
        std::ifstream inputFile(inputFilename);
        if (!inputFile) {
            throw std::runtime_error("Nie␣można␣otworzyć␣pliku␣wejściowego:␣" +
                inputFilename);
        }
```

Wczytywanie zawartości pliku do bufora

```
        std::stringstream buffer;
        buffer << inputFile.rdbuf();
        std::string content = buffer.str();
```

Zamiana „directed” na „undirected”

```
        size_t pos = content.find("<graph␣edgedefault=\"directed\">");
        if (pos != std::string::npos) {
            content.replace(pos, std::string("<graph␣edgedefault=\"directed\">").
                length(), "<graph␣edgedefault=\"undirected\">");
        }
```

Dodanie brakujących atrybutów

```
        if (content.find("<key␣attr.name=\"label\"") == std::string::npos) {
            content.insert(content.find("<graph␣"), "<key␣attr.name=\"label\"␣attr.
                type=\"string\"␣for=\"node\"␣id=\"label\"/>\n");
        }
        if (content.find("<key␣attr.name=\"weight\"") == std::string::npos) {
            content.insert(content.find("<graph␣"), "<key␣attr.name=\"weight\"␣attr.
                type=\"double\"␣for=\"edge\"␣id=\"weight\"/>\n");
        }
```

Usuwanie niepotrzebnych danych

```
        const std::vector<std::string> keysToRemove = {"b", "g", "r", "size"};
        for (const auto& key : keysToRemove) {
            size_t pos;
            while ((pos = content.find("<data␣key=\"" + key + "\">")) != std::string
                ::npos) {
                size_t endPos = content.find("</data>", pos);
                if (endPos != std::string::npos) {
                    content.erase(pos, endPos - pos + 7);
                }
            }
        }
```

Usuwanie linii zawierających `<data key="v_name">` i `<data key="v_type">`

```
        const std::vector<std::string> keysToDelete = {"v_name", "v_type"};
        for (const auto& key : keysToDelete) {
            size_t pos;
```

```
        while ((pos = content.find("<data␣key=\"" + key + "\">")) != std::string
            ::npos) {
            size_t lineStart = content.rfind("\n", pos);  // Znajdź początek
                linii
            if (lineStart == std::string::npos) {
                lineStart = 0;  // Jeśli nie ma poprzedzającej linii, to początek
                     pliku
            }

            size_t lineEnd = content.find("\n", pos);  // Znajdź koniec linii
            if (lineEnd == std::string::npos) {
                lineEnd = content.length();  // Jeżeli nie ma nowej linii, to
                    koniec pliku
            }

            content.erase(lineStart, lineEnd - lineStart + 1);  // Usuń całą lini
                ę
        }
    }
```

Usuwanie linii zawierających „node_color”, „node_size”, itd.

```
    const std::vector<std::string> additionalKeysToDelete = {
        "name", "group", "node_color", "node_size", "node_shape", "
            node_border_color", "node_border_width", "node_opacity", "
            node_position_x", "node_position_y", "node_weight", "node_type", "
            ip_address", "mac_address", "node_capacity", "node_bandwidth", "
            node_status", "description", "vlan_id", "node_utilization", "ports",
            "node_latency", "node_community", "edge_color", "edge_thickness", "
            edge_style", "edge_opacity", "edge_weight", "graph_description", "
            graph_version"
    };
    for (const auto& key : additionalKeysToDelete) {
        size_t pos;
        while ((pos = content.find("<data␣key=\"" + key + "\">")) != std::string
            ::npos) {
            size_t lineStart = content.rfind("\n", pos);  // Znajdź początek
                linii
            if (lineStart == std::string::npos) {
                lineStart = 0;  // Jeśli nie ma poprzedzającej linii, to początek
                     pliku
            }

            size_t lineEnd = content.find("\n", pos);  // Znajdź koniec linii
            if (lineEnd == std::string::npos) {
                lineEnd = content.length();  // Jeżeli nie ma nowej linii, to
                    koniec pliku
            }

            content.erase(lineStart, lineEnd - lineStart + 1);  // Usuń całą lini
                ę
        }
    }
```

Zamiana <data key="weight"> na <data key="label"> w przypadku znalezienia

```
    size_t lineStart = 0;
    while ((pos = content.find("<data␣key=\"weight\">", lineStart)) != std::
        string::npos) {
        size_t lineEnd = content.find("\n", pos);  // Znajdź koniec linii
        if (lineEnd == std::string::npos) {
            lineEnd = content.length();  // Jeżeli nie ma nowej linii, to koniec
                pliku
        }

        // Zamiana <data key="weight"> na <data key="label">
        content.replace(pos, std::string("<data␣key=\"weight\">").length(), "<
            data␣key=\"label\">");

        lineStart = lineEnd;  // Przesuń początek wyszukiwania do końca linii
    }
```

Zapisanie zmodyfikowanego pliku

```
        std::ofstream outputFile(outputFilename);
        if (!outputFile) {
            throw std::runtime_error("Nie␣można␣zapisać␣pliku␣wyjściowego:␣" +
                outputFilename);
        }
        outputFile << content;
```

3.6 LoadGraph - Wczytywanie grafu

Pierwsza część kodu czyści poprzednie dane grafu, a następnie sprawdza, czy podana nazwa pliku posiada rozszerzenie „ .graphml”. Jeśli nie, zgłaszany jest błąd. Następnie tworzona jest zmodyfikowana wersja pliku GraphML, aby zapewnić poprawne wczytanie danych. W tym celu wywoływana jest funkcja ModifyGraphmlFile, która modyfikuje oryginalny plik i zapisuje jego zmienioną wersję.

```
    size_t pos = filename.rfind(".graphml");
    if (pos == std::string::npos) {
        throw std::runtime_error("Plik␣nie␣ma␣rozszerzenia␣.graphml");
    }
    std::string modifiedFilename = filename.substr(0, pos) + "_modified.graphml";
    ModifyGraphmlFile(filename, modifiedFilename);
```

Kolejnym krokiem jest próba otwarcia zmodyfikowanego pliku. Jeśli operacja się nie powiedzie, zgłaszany jest błąd.

```
    std::ifstream file(modifiedFilename);
    if (!file) {
        throw std::runtime_error("Nie␣można␣otworzyć␣zmodyfikowanego␣pliku:␣" +
            modifiedFilename);
    }
```

Następnie tworzony jest obiekt dynamicznych właściwości, który przypisuje odpowiednie atrybuty do wierzchołków i krawędzi grafu. Dla wierzchołków przypisane są:

- label (etykieta wierzchołka)
- x (współrzędna X wierzchołka)
- y (współrzędna Y wierzchołka) Dla krawędzi przypisana jest właściwość:
- label (etykieta krawędzi)

```
    boost::dynamic_properties dp;
    dp.property("label", boost::get(&VertexProperties::label, g));
    dp.property("x", boost::get(&VertexProperties::x, g));
    dp.property("y", boost::get(&VertexProperties::y, g));
    dp.property("label", boost::get(&EdgeProperties::label, g));
    boost::read_graphml(file, g, dp);
```

Po wczytaniu grafu kod dodaje brakujące krawędzie odwrotne, jeśli nie istnieją, zapewniając, że każda krawędź ma swoje odbicie w przeciwnym kierunku.

```
    for (auto e : boost::make_iterator_range(boost::edges(g))) {
        auto source = boost::source(e, g);
        auto target = boost::target(e, g);
        if (!boost::edge(target, source, g).second) {
            boost::add_edge(target, source, g);
        }
    }
```

Następnie kod zapewnia, że współrzędne wierzchołków oraz wagi krawędzi mają minimalne wartości.

```
    for (auto v : boost::make_iterator_range(vertices(g))) {
        auto& vertex = g[v];
        vertex.x = std::max(0.0f, vertex.x);
        vertex.y = std::max(0.0f, vertex.y);
```

```
        }
        for (auto e : boost::make_iterator_range(edges(g))) {
            auto& edge = g[e];
            edge.weight = std::max(1, edge.weight);
        }
```

Współrzędne wierzchołków są skalowane do obszaru 800x600, aby poprawić ich wizualizację.

```
        ScaleCoordinates(g, 800, 600);
```

Na końcu plik tymczasowy jest zamykany i usuwany. Jeśli usunięcie pliku nie powiedzie się, zgłaszany jest błąd.

```
        file.close();
        if (std::remove(modifiedFilename.c_str()) != 0) {
            throw std::runtime_error("Błąd przy usuwaniu pliku: " + modifiedFilename);
        }
```

Jak widać, decyzja o modyfikacji pliku wprowadza pewne ograniczenia dla naszego programu. Niemniej jednak, podjęliśmy tę decyzję, licząc na potencjalne korzyści. Biblioteka Boost zawiera funkcje, które miały nam pomóc, między innymi w rysowaniu grafu oraz obliczaniu najkrótszej ścieżki. Jednak, jak się okazało, użycie tych funkcji nie było tak proste, jak początkowo zakładaliśmy. Trudności pojawiły się podczas próby znalezienia odpowiedzi na wiele pytań w bardzo skomplikowanej dokumentacji technicznej. Co za tym idzie wczytywanie danych za pomocą Boost okazało się błędem. W przypadku ręcznego wczytywania pliku nasz program mógłby posiadać jeszcze więcej funkcji. Warto też zaznaczyć, że program nie wczyta każdego grafu, ponieważ może on zawierać niestandardowe lub nieoczekiwane atrybuty, struktury, lub rozszerzenia, które nie zostały uwzględnione w implementacji (a z którymi funkcja z biblioteki boost ma problem), co może prowadzić do błędów podczas przetwarzania. Warto zaznaczyć, że nasz program w przypadku nie wczytania danego grafu, zawsze zwróci konkretny błąd (zwróci informacje dlaczego danego pliku nie może wczytać). Mimo to, funkcja uwzględnia występowanie ponad 40 najczęściej spotykanych atrybutów, takich jak „`node_color`”, „`node_size`”, „`edge_weight`”, czy „`label`”, co czyni ją bardzo wszechstronną w praktycznych zastosowaniach. Warto zauważyć, że w plikach GraphML teoretycznie może występować dowolna liczba atrybutów, ponieważ GraphML pozwala na definiowanie własnych kluczy i struktur danych. Nie sposób przewidzieć i uwzględnić wszystkich możliwych kombinacji. Na dowód poniżej znajduje się link do strony opisującej jak definiować niestandardowe klucze w plikach graphml yworks.com Obecna implementacja stanowi kompromis pomiędzy prostotą a funkcjonalnością.

3.7 Rysowanie grafu

Sprawdzanie, czy graf jest pusty

```
        if (boost::num_vertices(g) == 0) {
            wxMessageBox("Najpierw wczytaj plik!", "Uwaga", wxOK | wxICON_WARNING);
            return;
        }
```

Sprawdzanie, która opcja jest zaznaczona(), aby ustalić tryb rysowania

```
        int mode = RadioBox1->GetSelection() + 1;
```

Sprawdzanie, czy wybrano sposób rysowania grafu

```
        if (mode == 0) {
            wxMessageBox("Wybierz sposób rysowania grafu!", "Uwaga", wxOK |
                wxICON_WARNING);
            return;
        }
```

Tworzenie nowego okna do rysowania grafu i jego wyświetlanie

```
        GraphWindow* graphWindow = new GraphWindow(this, g, mode);
        graphWindow->Show(true);
```

3.8 Wyświetlanie wierzchołków

Proces wizualizacji wierzchołków odbywa się przy użyciu pętli `for`, która iteruje przez wszystkie wierzchołki grafu. Dla każdego wierzchołka odczytywane i wyświetlane są jego atrybuty, takie jak etykieta oraz pozycja:

```
        for (auto v : boost::make_iterator_range(boost::vertices(g))) {
            const auto& vertex = g[v];
            oss << "Wierzchołek: " << vertex.label
            << ", Pozycja: (" << vertex.x << ", " << vertex.y << ")\n";
        }
```

3.9 Wyświetlanie krawędzi

Analogicznie, proces wyświetlania krawędzi opiera się na pętli `for`, która iteruje przez wszystkie krawędzie grafu. Dla każdej krawędzi pobierane i wyświetlane są jej atrybuty, a także informacje o wierzchołkach źródłowym i docelowym:

```
        for (auto e : boost::make_iterator_range(boost::edges(g))) {
            const auto& edge = g[e];
            const auto& source = g[boost::source(e, g)];
            const auto& target = g[boost::target(e, g)];

            oss << "Krawędź: " << edge.label
            << ", Od: " << source.label
            << ", Do: " << target.label
            << "\n";
        }
```

3.10 Sprawdzanie typu grafu

Sprawdzanie, czy graf jest pełny:

Graf jest pełny, jeśli liczba krawędzi równa się maksymalnej liczbie krawędzi w grafie prostym:

```
    is_complete = edge_count == (vertex_count * (vertex_count - 1)) / 2;
```

Sprawdzanie spójności grafu:

Graf jest spójny, jeśli liczba składowych wynosi 1. Liczba spójnych składowych jest obliczana za pomocą funkcji `boost::connected_components`:

```
    int num_components = boost::connected_components(g, &component_map[0]);
    bool is_connected = num_components == 1;
```

3.11 Sprawdzanie cykliczności grafu:

Graf jest cykliczny, jeśli liczba krawędzi przekracza liczbę wierzchołków minus 1. Do dokładnej weryfikacji został użyty algorytmu BFS, oraz inne przydatne do sprawdzania cykliczności grafu funkcje z boost:

```
        if (edge_count > vertex_count - 1) {
            boost::breadth_first_search(g, *boost::vertices(g).first,
            boost::visitor(boost::bfs_visitor<boost::null_visitor>())
            .color_map(boost::make_iterator_property_map(color_map.begin(), get(boost::
                vertex_index, g))));
        }
```

Sprawdzanie dwudzielności grafu:

Dwudzielność jest sprawdzana za pomocą algorytmu BFS, który przypisuje kolory wierzchołkom. Jeśli dwa sąsiadujące wierzchołki mają ten sam kolor, graf nie jest dwudzielny:

```
        if (color_map[neighbor] == color_map[u]) { is_bipartite = false; break; }
```

Sprawdzanie, czy graf jest drzewem lub lasem:

Graf jest drzewem, jeśli jest spójny i ma dokładnie `vertex_count` - 1 krawędzi. Jest lasem, jeśli nie jest cykliczny:

```
        is_tree = (is_connected && edge_count == vertex_count - 1);
        is_forest = !is_cyclic;
```

3.12 Sprawdzanie planarności grafu:

Planarność grafu jest weryfikowana za pomocą testu Boyera-Myrvolda, wykorzystując funkcję z biblioteki boost:

```
        is_planar = boost::boyer_myrvold_planarity_test(g);
```

3.13 Wyszukiwanie najkrótszej ścieżki:

Do znajdowania najkrótszej ścieżki wykorzystany został algorytm Dijkstry. A przycisk działa w skrócie w następujący sposób:

Sprawdzenie, czy graf jest pusty:

Jeśli graf nie zawiera wierzchołków, funkcja wyświetla komunikat o błędzie i kończy działanie.

```
        if (boost::num_vertices(g) == 0) {
            wxMessageBox("Najpierw wczytaj plik!", _("Błąd"), wxOK | wxICON_WARNING);
            return;
        }
```

Pobranie etykiety wierzchołka początkowego:

Użytkownik wprowadza nazwę wierzchołka początkowego poprzez okno dialogowe.

```
        wxTextEntryDialog dialog(this, _("Podaj etykietę wierzchołka początkowego:"), _("
            Najkrótsze ścieżki"));
        if (dialog.ShowModal() != wxID_OK) {
            return;
        }
        std::string startLabel = std::string(dialog.GetValue().mb_str());
```

Znajdowanie wierzchołka o podanej etykiecie:

Iteracja przez wszystkie wierzchołki w celu znalezienia tego o podanej etykiecie.

```
        boost::graph_traits<Graph>::vertex_descriptor startVertex;
        bool found = false;

        for (auto v : boost::make_iterator_range(boost::vertices(g))) {
            if (g[v].label == startLabel) {
                startVertex = v;
                found = true;
                break;
            }
        }
```

```
        if (!found) {
            wxMessageBox(_("Nie znaleziono wierzchołka o podanej etykiecie!"), _("Błąd"),
                 wxOK | wxICON_ERROR);
            return;
        }
```

Inicjalizacja struktur danych

`distances` — mapa przechowująca minimalną liczbę krawędzi do każdego wierzchołka (domyślnie ∞).
`predecessors` — mapa przechowująca poprzednika na najkrótszej ścieżce.
`pq` — kolejka priorytetowa do przetwarzania wierzchołków.

```
            std::unordered_map<boost::graph_traits<Graph>::vertex_descriptor, int>
                distances;
            std::unordered_map<boost::graph_traits<Graph>::vertex_descriptor, boost::
                graph_traits<Graph>::vertex_descriptor> predecessors;
            std::priority_queue<std::pair<int, boost::graph_traits<Graph>::
                vertex_descriptor>,
            std::vector<std::pair<int, boost::graph_traits<Graph>::vertex_descriptor>>,
            std::greater<>> pq;

            for (auto v : boost::make_iterator_range(boost::vertices(g))) {
                distances[v] = std::numeric_limits<int>::max();
            }

            distances[startVertex] = 0;
            pq.push({0, startVertex});
```

Algorytm Dijkstry:

Pobieranie wierzchołka o najmniejszej liczbie krawędzi. Dla każdego sąsiada sprawdzenie, czy nowa ścieżka jest krótsza. Wagi krawędzi ustawione na 1, więc optymalizujemy liczbę przejść.

```
        while (!pq.empty()) {
            auto [dist, currentVertex] = pq.top();
            pq.pop();

            if (dist > distances[currentVertex]) {
                continue;
            }

            for (auto edge : boost::make_iterator_range(boost::out_edges(currentVertex, g
                ))) {
                auto neighbor = boost::target(edge, g);
                int weight = 1; // Każda krawędź ma wagę 1, liczymy liczbę krawędzi

                if (distances[currentVertex] + weight < distances[neighbor]) {
                    distances[neighbor] = distances[currentVertex] + weight;
                    predecessors[neighbor] = currentVertex;
                    pq.push({distances[neighbor], neighbor});
                }
            }
        }
```

Wyświetlenie wyników:

Iteracja przez wszystkie wierzchołki i wypisanie minimalnej liczby krawędzi. Jeśli wierzchołek jest nieosiągalny, zwracana jest informacja „Brak połączenia”.

```
        std::ostringstream oss;
        oss << "Najkrótsze ścieżki od wierzchołka: " << startLabel << "\n\n";
        for (auto v : boost::make_iterator_range(boost::vertices(g))) {
            oss << "Do wierzchołka " << g[v].label << ": ";
            if (distances[v] == std::numeric_limits<int>::max()) {
```

```
                oss << "Brak połączenia\n";
            } else {
                oss << distances[v] << " (liczba krawędzi)\n";
            }
        }

        wxMessageBox(wxString::FromUTF8(oss.str().c_str()), _("Najkrótsze ścieżki"), wxOK
             | wxICON_INFORMATION
```

Program implementuje algorytm Dijkstry do obliczania najkrótszych ścieżek w grafie, wykorzystując bibliotekę Boost i wxWidgets. Umożliwia użytkownikowi wprowadzenie etykiety wierzchołka początkowego, a następnie oblicza najkrótsze ścieżki do wszystkich pozostałych wierzchołków, uwzględniając liczbę krawędzi. Program wyświetla wyniki w postaci liczby krawędzi dla każdego wierzchołka lub informuje o braku połączenia. Istnieje potencjał na rozwój programu poprzez dodanie obsługi wag krawędzi w prosty sposób, w momencie gdy w przyszłości zostanie zmodyfikowany proces wczytywania grafu.

4 Podsumowanie

Program stanowi użyteczne narzędzie do analizy grafów zapisanych w formacie GraphML, oferując szereg funkcji, które umożliwiają użytkownikom łatwą wizualizację oraz ocenę kluczowych właściwości grafów. Użytkownicy mogą przeprowadzać analizy takie jak sprawdzanie spójności, wykrywanie cykliczności, czy sprawdzanie planarności grafu.
Podczas tworzenia projektu zdobyliśmy cenne doświadczenie, które pozwoliło nam zastosować wiedzę zdobytą na zajęciach w praktyce.

5 Podział pracy

- Interfejs – Maja Darczuk
- Tworzenie tła – Maja Darczuk
- Rysowanie grafu – Maja Darczuk
- Przycisk do rysowania grafu – Maja Darczuk
- Przycisk do wyświetlania typu grafu – Maja Darczuk
- Skalowanie grafu – Jan Walkiewicz
- Modyfikacja grafu - Jan Walkiewicz
- Przycisk do wyświetlania wierzchołków – Jan Walkiewicz
- Przycisk do wyświetlania krawędzi – Jan Walkiewicz
- Przycisk do wyświetlania najkrótszej ścieżki – Jan Walkiewicz
- Definicja Grafu – 50% – Maja Darczuk, 50% – Jan Walkiewicz
- Przycisk do wczytania grafu – 30% – Maja Darczuk, 70% – Jan Walkiewicz
- Sprawozdanie – 70% – Maja Darczuk, 30% – Jan Walkiewicz

Źródła pomocnicze

[1] Strona z opisem bibliotek boost: boost.org

[2] Przykładowe algorytmy do opisywania grafów w C++: geeksforgeeks.org

[3] „Programowanie wieloplatformowe z C++ i wxWidgets 3” Bartosz W. Warzocha